

Object-Oriented Programming

TYPE CASTING & CONSTRUCTORS

Marvin C Santos

Instructor

TYPE CASTING



- ▶ It's when you assign a value of one data type to another type.

WHAT IS TYPE CASTING?

- ▶ To ensure a function handles the variables correctly.
- ▶ C# is a compiled language
- ▶ To treat our data properly
- ▶ To guarantee the functionality

WHY WE NEED TO USE TYPE CASTING?

SAMPLE:

```
string sentByMail_var1 = "Marvin Santos";  
string sentByMail_var2 = "300";  
int loanBalance = 2000;  
int totalLoan = loanBalance + sentByMail_var2;
```

```
int totalLoan = loanBalance + sentByMail_var2;
```



[🔗] (local variable) string sentByMail_var2

'sentByMail_var2' is not null here.

CS0029: Cannot implicitly convert type 'string' to 'int'

Show potential fixes (Alt+Enter or Ctrl+.)



DATA TYPES SIZES



Bytes
1
1
2
2
4
4
8
8
4
8
24
2

Data Type	Range
byte	0 .. 255
sbyte	-128 .. 127
short	-32,768 .. 32,767
ushort	0 .. 65,535
int	-2,147,483,648 .. 2,147,483,647
uint	0 .. 4,294,967,295
long	-9,223,372,036,854,775,808.. 9,223,372,036,854,775,807
ulong	0 .. 18,446,744,073,709,551,615
float	-3.402823e38 .. 3.402823e38
double	-1.79769313486232e308.. 1.79769313486232e308
decimal	-79228162514264337593543950335 to 79228162514264337593543950335
char	A Unicode character.
string	A string of Unicode characters.
bool	True or False.
object	An object.

► Implicit Casting

- Converting a smaller type to a larger type.
(*byte - int - long - double*)

► Explicit Casting

- Converting a larger type to a smaller type.
(*double - long - int - byte*)

2 TYPES OF CASTING IN C#

```
//bit Binary 1 & 0  
//1 byte = 8bit = 1111 1111 = 255
```

```
byte a;  
a = 255;
```

```
byte b;  
b = 256;
```

```
//int = 4 bytes  
// 1111 1111,1111 1111,  
// 1111 1111,1111 1111,
```

```
int mySalary;  
mySalary = 2500;
```

```
//implicit  
mySalary = a;
```

```
Console.WriteLine("Implicit: " + mySalary);
```

NOT VALID

```
// NOT passible  
a = mySalary;
```

NOT VALID

```
// NOT possible  
a = mySalary;
```

HERE COMES **EXPLICIT** CONVERSION:

```
a = (byte)mySalary;
```

WHAT WILL BE THE VALUE OF A?

```
a = (byte)mySalary;  
Console.WriteLine("explicit: " + a);
```

```
explicit: 255
```

- ▶ int to string
- ▶ int to double
- ▶ double to int
- ▶ boolean to string

OTHER CONVERSION TYPE

OTHER CONVERSION TYPE

```
int y = 20;  
double z = 74.5;  
bool yesNo = true;
```

```
//convert int to string  
Console.WriteLine("Integer to string: " + Convert.ToString(y));
```

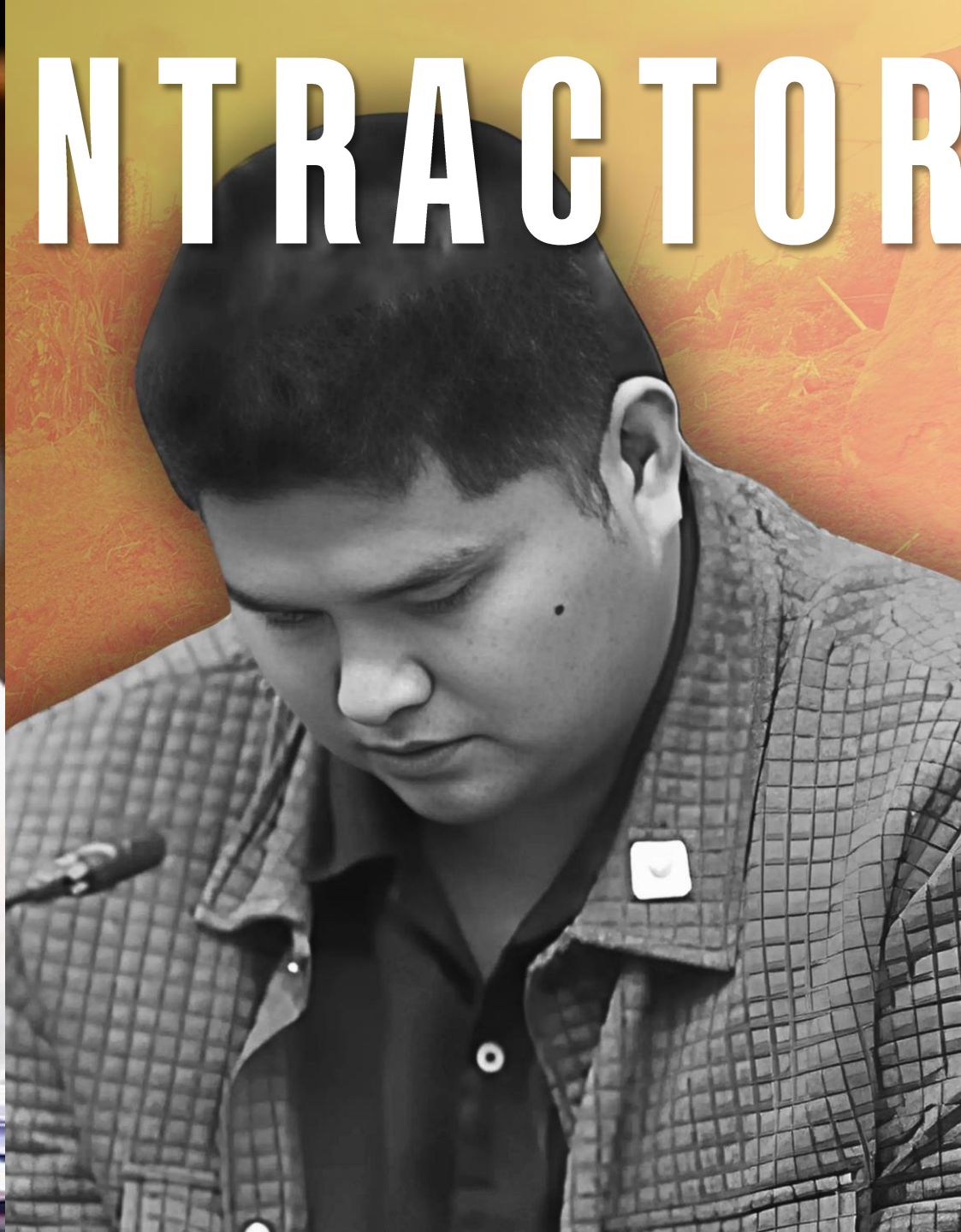
```
//int to double  
z = Convert.ToDouble(y);  
Console.WriteLine("Integer to double: " + z);
```

```
//double to int  
y = Convert.ToInt32(z);  
Console.WriteLine("Double to Integer: " + y);
```

```
//boolean to string  
Convert.ToString(yesNo);  
Console.WriteLine("Boolean to String: " + Convert.ToString(yesNo));
```




CONTRACTOR



CONSTRUCTORS



WHAT ARE CONSTRUCTORS?

- ▶ These are **special methods** in C# that will invoke automatically when an object of a class is created.

NAMING A CONSTRUCTORS

- ▶ Constructors have the same name as the Class
- ▶ It does not have a return type, not even void.

IMPLEMENTING CONSTRUCTORS



- ▶ “When you create a default constructor, you can SET an initial value of the class itself”

IMPLEMENTING DEFAULT CONSTRUCTOR

► “Implementing Constructors in different ways.”

POLYMORPHISM ON CONSTRUCTORS

▶ “Constructors that copy itself”

COPY CONSTRUCTORS

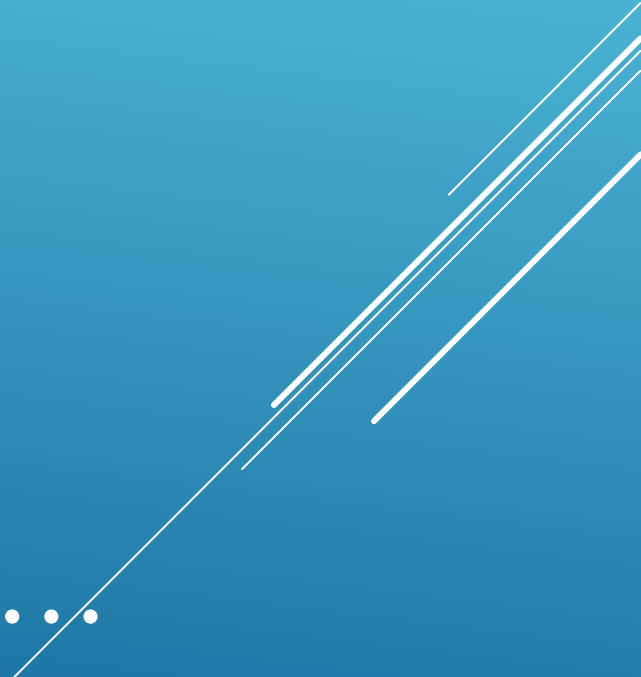
▶ “Preventing to implement this specific constructor, while implementing Encapsulation”

PRIVATE CONSTRUCTORS

► “You can only invoke this static Constructor...ONCE”

STATIC CONSTRUCTOR

THANK YOU...

Several thin, parallel white lines of varying lengths and orientations are positioned in the bottom right corner of the slide, creating a modern, geometric design element.